

DD2459 Software Reliability 2021
Take-home Exam
Monday 15 March, 15.00 – Wednesday 17 March, 12.00

Instructions.

Please read the following instructions carefully.

- (1) Clearly mark at the top of each sheet you use: (a) your name, (b) the page number.
- (2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)
- (3) Your work should be uploaded to Canvas, no later than Wednesday 17 March 2021 at 12.00 midday. After this time, it will be marked as late and points will be subtracted.
- (4) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm! You can also e-mail the examiner at karlm@kth.se.
- (5) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates to this document.
- (6) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) *By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page.* (d) In the case that cheating is suspected, disciplinary action will be taken.
- (7) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

Please turn over.

NOTE: For a full score of 80 points you should answer all 4 questions below.

Question 1. (Total 15 points)

Write appropriate pre- and post-conditions for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid under-specification, i.e. write all the constraints necessary for each condition and do not omit any relevant constraints.

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book.

You do not have to provide Java code for any of the operations, only JML.

(i) **(5 points)** A *factorial* function:

```
int factorial(int x)
```

Input: One positive integer parameter x.

Output: The integer result which should be x!

Important Note: Since most JML tools cannot handle recursive JML models you should not use a recursive solution such as: `\result == x * factorial(x-1)`

Model Answer:

We could make use of Stirlings approximation (or any other?) and write

```
requires x >= 0
```

```
ensures \old(x) == 0 ==> \result == 1 &  
\old(x) > 0 ==>
```

```
Java.Lang.Math.abs(\result -  
Java.Lang.Math.sqrt(2 * Java.Lang.Math.pi * n) * (n/e)^n )  
< epsilon
```

and epsilon is some small constant value. Another exact solution is:

```
requires x >= 0
```

```
ensures \old(x) == 0 ==> \result == 1 &  
\old(x) > 0 ==> \result == (\product int i; 1 <= i & i <=  
x; i )
```

(ii) **(5 points)** A *Fibonacci* function:

```
int Fibb(int n)
```

Input: One positive integer parameter n.

Output: The nth number in the Fibonacci series.

Important Note: Since most JML tools cannot handle recursive JML models you should not use a recursive solution such as: `\result == Fibb(n-1) + Fibb(n-2)`

Model Answer:

Since the Fibonacci series is an example of a linear recurrence with constant coefficients, then it has a closed form solution known as *Binet's formula*:

requires $n \geq 0$

ensures $\text{Java.Lang.Math.abs}(\text{result} - B^n - (-B)^{-n} / (2B - 1)) < \text{epsilon}$

where $B = (1 + \text{Java.Lang.Math.sqrt}(5))/2$
and epsilon is some small constant value.

An exact solution without Binet's formula would be

requires $n \geq 0$

ensures $n == 0 \implies \text{result} == 0 \ \&$
 $n == 1 \implies \text{result} == 1 \ \&$
 $n > 1 \implies (\exists \text{int}[] \text{fibseries}; \text{fibseries.length} == n;$
 $\text{fibseries}[0] == 0 \ \&$
 $\text{fibseries}[1] == 1 \ \&$
 $(\forall \text{int } i; 2 \leq i \ \& \ i < \text{fibseries.length};$
 $\text{fibseries}[i] == \text{fibseries}[i-1] + \text{fibseries}[i-2])$
 $\ \& \ \text{result} == \text{fibseries}[n]$

Another approach is to declare a model variable

```
@public model int[] fibseries
```

requires $n \geq 0$

ensures $n == 0 \implies \text{result} == 0 \ \&$
 $n == 1 \implies \text{result} == 1 \ \&$
 $n > 1 \implies \text{fibseries.length} == n \ \&$
 $\text{fibseries}[0] == 0 \ \&$
 $\text{fibseries}[1] == 1 \ \&$
 $(\forall \text{int } i; 2 \leq i \ \& \ i < \text{fibseries.length};$
 $\text{fibseries}[i] == \text{fibseries}[i-1] + \text{fibseries}[i-2])$
 $\ \& \ \text{result} == \text{fibseries}[n]$

- (iii) **(5 points)** A *Greatest Common Divisor (GCD) function*:

```
int GCD(int x, int y)
```

Input: Two integer parameters `x`, `y`, not both 0.

Output: The greatest common divisor (GCD) of `x` and `y`.

Model Answer:

```
requires x > 0 | y > 0

ensures x%\result == 0 & y%\result == 0
// \result is a divisor of both x and y
& (\forall int d; 0 <= d & x <= y ==> d <= y & y <= x
==> d <= x;
x%d == 0 & y%d == 0 ==> %\result >= d)
// \result is the greatest of all the divisors of both
x and y
```

Question 2. (Total 5 points)

- (i) **(3 points)** Write down one metamorphic equation that should be satisfied by the arithmetic average function:

```
float average(float[] distribution)
```

Input: A one-dimensional array `distribution` of `float` (the data set).

Output: The numerical average of the values in `distribution`.

Clearly define the data transformation used on the input in your equation.

Model Answer: The average function satisfies any permutative law (similar to the inclusive law mentioned in Lecture 6) such as

average (`[x1, ..., xn]`) = **average** (`T([x1, ..., xn])`)

where

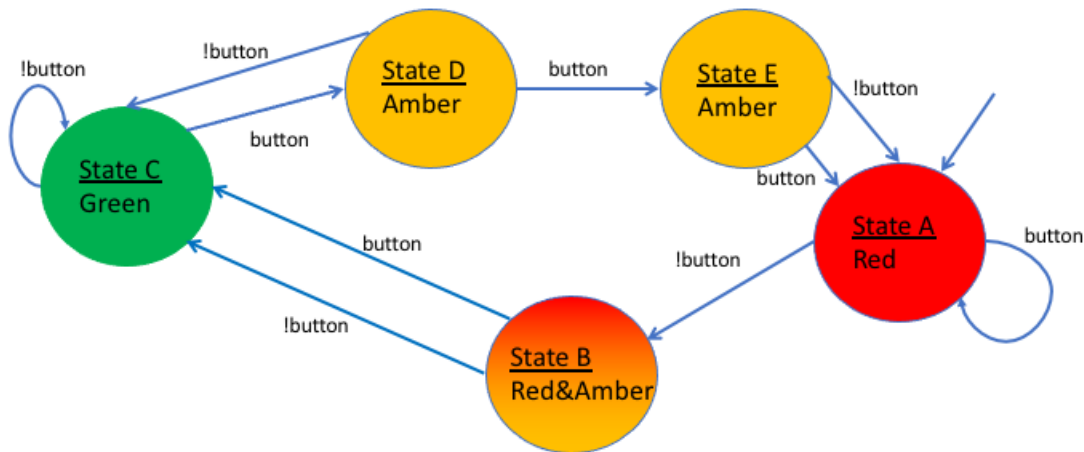
`T ([x1, ..., xn]) = [xn, ..., x1]`

- (ii) **(2 points)** Starting from any concrete seed test case (i.e. input array values), write down a second test case using your data transformation defined in your answer to part (i).

Model Answer: Starting from the seed test case $[0.0, 1.0, , \dots, n.0]$, the second test case would be $T([0.0, 1.0, , \dots, n.0]) = [n.0, n-1.0, , \dots, 0.0]$.

Question 3. (Total 21 points) The diagram below shows a five state finite state machine (FSM) model of a *pedestrian road crossing at a traffic light*. The traffic light can give one of four different output values (light colours) as signals to vehicles. The system has a single Boolean input value *button*, which can be true or false at any time. The transitions for each state are shown in the diagram.

Pedestrian Road Crossing Finite State Machine



(a) (3 points) Write down a set of linear temporal logic (LTL) trap properties that would achieve *full node coverage* (NC) when generating test cases as counterexamples to the properties. You do not need to eliminate redundant requirements.

Model Answer: without eliminating redundancy - 5 trap properties, notice that State is an variable of type enumeration while Button is a variable of type Boolean.

$G(\neg \text{State} = A), G(\neg \text{State} = B), G(\neg \text{State} = C), G(\neg \text{State} = D), G(\neg \text{State} = E)$

(b) (5 points) Write down a set of linear temporal logic (LTL) trap properties that would achieve *full edge coverage* (EC) when generating test cases as counterexamples to the properties. You do not need to eliminate redundant requirements.

Model Answer: without eliminating redundancy - 10 trap properties

$G(\text{State} = A \ \& \ \text{button} \rightarrow \neg X(\text{state} = A)), G(\text{State} = A \ \& \ \neg \text{button} \rightarrow \neg X(\text{state} = B))$

$G(\text{State} = B \ \& \ \text{button} \rightarrow !X(\text{state} = C)) , G(\text{State} = B \ \& \ !\text{button} \rightarrow !X(\text{state} = C))$

$G(\text{State} = C \ \& \ \text{button} \rightarrow !X(\text{state} = D)) , G(\text{State} = C \ \& \ !\text{button} \rightarrow !X(\text{state} = C))$

$G(\text{State} = D \ \& \ \text{button} \rightarrow !X(\text{state} = E)) , G(\text{State} = D \ \& \ !\text{button} \rightarrow !X(\text{state} = C))$

$G(\text{State} = E \ \& \ \text{button} \rightarrow !X(\text{state} = A)) , G(\text{State} = B \ \& \ !\text{button} \rightarrow !X(\text{state} = A))$

(c) **(8 points)** Write down a set of linear temporal logic (LTL) trap properties that would achieve full edge-pair coverage ($EPC = EC^2$) when generating test cases as counterexamples to the properties. You do not need to eliminate redundant requirements, but you should try to make each requirement as logically simple as possible.

Model Answer: without eliminating redundancy - 20 trap properties

$G(\text{State} = A \ \& \ \text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = A)) , G(\text{State} = A \ \& \ \text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = B))$

$G(\text{State} = A \ \& \ !\text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = C)) , G(\text{State} = A \ \& \ !\text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = C))$

$G(\text{State} = B \ \& \ \text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = D)) , G(\text{State} = B \ \& \ \text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = C))$

$G(\text{State} = B \ \& \ !\text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = D)) , G(\text{State} = B \ \& \ !\text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = C))$

$G(\text{State} = C \ \& \ \text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = E)) , G(\text{State} = C \ \& \ \text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = C))$

$G(\text{State} = C \ \& \ !\text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = D)) , G(\text{State} = C \ \& \ !\text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = C))$

$G(\text{State} = D \ \& \ \text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = A)) , G(\text{State} = D \ \& \ \text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = A))$

$G(\text{State} = D \ \& \ !\text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = D)) , G(\text{State} = D \ \& \ !\text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = C))$

$G(\text{State} = E \ \& \ \text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = A)) , G(\text{State} = E \ \& \ \text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = B))$

$G(\text{State} = E \ \& \ !\text{button} \ \& \ X(\text{button}) \rightarrow !XX(\text{state} = A)) , G(\text{State} = E \ \& \ !\text{button} \ \& \ X(!\text{button}) \rightarrow !XX(\text{state} = B))$

(d) ^[11]_[SEP] **(5 points)** An FSM A (such as the Pedestrian Road Crossing FSM model above) can be said to be fair with respect to any state s of A if the LTL formula

$$G(F(\text{state} = s))$$

is a *valid LTL formula* for A . Recall that an LTL formula f is valid for A if, and only if, there is no counterexample $i = (i_1, i_2, i_3, \dots)$ (an infinite input sequence for A) that makes f *false*.

(d.i) Explain *in words*, precisely what the above fairness formula means in terms of state s .

Model Answer: An FSM *A* can be said to be fair wrt. a state *s* (which must be a state of *A*) if whatever current state we are in, we can eventually get to the state *s* after some finite input or time. In particular this means *s* is reachable from every other reachable state *s'*.

(d.ii) *Discuss* whether the Pedestrian Road Crossing FSM above is fair for each of its 5 states. Motivate your answer.

Model Answer: For each state B, C, D, E the PRC FSM is not fair since the infinite input sequence,

(button, button, button,)

keeps PRC in state A forever. Finally, for state A, the infinite input

(!button, !button, !button,)

Keeps PRC in state C forever, so that A is never reached. Thus, PRC is not fair for any state.

(d.iii) Is it possible to *demonstrate* that a system is unfair (using the definition above) by software testing? *Motivate* your answer and discuss any *alternative approaches* to the problem.

Model Answer: No, it is not possible to demonstrate that a system is unfair by testing, since every counterexample must be infinite (the state *s* is never ever reached). We cannot execute an infinite failing test case in finite time. Said differently, after any finite time of failing to observe *s*, there is still the possibility that we might see *s* sometime later. Thus, the experiment to observe *s* cannot be stopped with a definitive “no” answer.

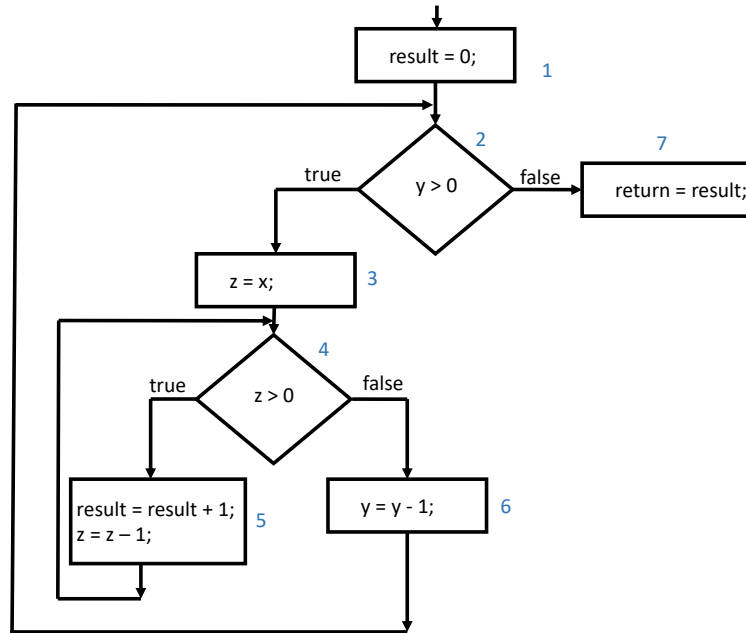
An alternative approach that could work to demonstrate unfairness is static analysis, e.g. by model checking. This approach is actually used in practice to establish fairness properties. Thus, static analysis can go beyond the reasoning power of testing.

Question 4. (Total 39 points) Consider the following ten lines of pseudo-code:

```
1.   result = 0;
2.   while (y > 0) do {
3.       z = x;
4.       while (z > 0) do {
5.           result = result + 1;
6.           z = z - 1;
7.       }
8.       y = y - 1;
9.   }
10.  return result;
```

You can assume that `x:integer` and `y:integer` are the only integer input variables to the above pseudo-code.

- (a) **(2 points)** Draw a condensation graph for the above pseudo-code. Clearly number all the nodes in your graph.



- (b) **(10 points)** Show that for your condensation graph in answer to part (a) that node coverage (NC) is the same as edge coverage (EC), i.e. let TS be any test suite for your condensation graph, then TS satisfies 100% node coverage if, and only if, TS satisfies 100% edge coverage.

Answer: Proof. Let TS be any test suite, i.e., a set of test cases for this condensation graph.

Suppose TS satisfies 100% EC. We must show that TS satisfies 100% NC. This is fairly obvious. Consider any node n (apart from the last node 7). By inspection of the graph, we see that there exists either an edge (n, v) or (v, n) for some node v . By assumption of 100% EC, there exists some test case tc in TS such that tc satisfies (n, v) or (v, n) . Hence n is satisfied by tc . Since n was arbitrarily chosen, then for every node in the graph, the requirement n is satisfied by some tc in TS. Thus, TS satisfies 100% NC.

Conversely, suppose TS satisfies 100% NC. We must show that TS satisfies 100% EC. This is a little harder. For each assignment block node $y = 1, 3, 5, 6, 7$, the node y has a single-entry edge (with the exception of $y = 1$) and a single exit edge (with the exception of $y = 7$), i.e., the edges (x, y) and (y, x) are unique. For these edges (x, y) , (y, x) , by assumption, some test case tc in TS satisfies the node requirement y . So then tc satisfies both of the edge requirements (x, y) and (y, x) .

Since every edge in the graph is the entry or exit edge of some assignment block then for every edge e there exists a test case tc in TS such that tc satisfies e . Thus, TS satisfies 100% EC.

- (c) (5 points) Use your reasoning for part (b) to define a general graph theoretic property P of condensation graphs such that: for any condensation graph G satisfying P , NC and EC always give the same coverage of G . Show that there exists at least one condensation graph G that does not satisfy P , and for this graph G , NC does not give the same coverage as EC . Hence conclude that in general NC is not the same as EC .

Answer: Let G be any condensation graph such that every edge is the exit or entry edge of an assignment block, and each assignment block has at most one entry edge. Then the general argument of Part (b) shows that NC and EC give the same coverage.

The lecture notes for Lecture 2 give an example of a graph with an assignment block having 2 entry edges. So, this graph fails the above property and for this graph, NC is not equal to EC .

- (d) (11 points) For the condensation graph of part (a), write down a control flow test requirement tr such that a test case tc satisfies your test requirement tr if, and only if, tc executes the body of each while loop exactly once? (**Clarification:** the two while loop bodies are the code sections contained in parentheses $\{ \}$ which start at lines 3 and 5 respectively in the pseudo-code.) How many test cases exist for this glass-box test requirement? Carefully explain your reasoning.

Answer: The control flow test requirement to execute each loop body exactly once is $P = 1, 2, 3, 4, 5, 4, 6, 2, 7$

We build a constraint table to find a general solution to this path as a constraint on the input variables x and y .

Node	Instruction	Constraints
1	result = 0	result' = 0
2	y > 0	y' > 0 $\Leftrightarrow$ y > 0
3	z = x	z' = x
4	z > 0	z' > 0 & z' = x & y > 0 $\Leftrightarrow$ x > 0 & y > 0
5	result = result+1	result' = 0+1
5	z = z - 1	z' = x - 1
4	z > 0	!(z' > 0) & z' = x - 1 & x > 0 & y > 0 $\Leftrightarrow$!(x > 1) & x > 0 & y > 0 $\Leftrightarrow$ x = 1 & y > 0
6	y = y - 1	y' = y - 1
2	y > 0	!(y' > 0) & y' = (y-1) & y > 0 & x = 1 $\Leftrightarrow$!(y > 1) & y > 0 & x = 1 $\Leftrightarrow$

		y = 1 & x = 1
7	return result	result' = 1

So, by row 9 (node 2) in the constraint table above there is a unique test case which traverses each loop body just once, namely $x = 1, y = 1$. Clearly this single test case gives 100% node coverage, and by (b) above it also gives 100% edge coverage.

- (e) (6 points) For the condensation graph of part (a), give as many edge pair requirements as you can, each of which demonstrates that 100% edge pair coverage (E^2C) gives greater coverage than 100% edge coverage (EC) and 100% node coverage (NC). Clearly explain and motivate each answer, and discuss any redundancy among them.

Answer: In general, edge coverage will give us coverage of some consecutive pairs of edges such as (1,2)(2,3). But not all pairs need be covered by a requirement set satisfying 100% EC in this way.

The question asks us to demonstrate edge pairs that would not necessarily be covered by edge coverage. For this, we can take a minimal requirement set that just achieves 100% EC such as

$P = 1, 2, 3, 4, 5, 4, 6, 2, 7$

and find out what edge pairs are missing.

Three edge pairs demonstrate the extended coverage of (E^2C) over P . There is no redundancy among them, as each can be satisfied without satisfying any of the others. So, we need to list out all three.

The edge pair (5, 4), (4, 5) which gives at least 2 executions of the inner loop body is not satisfied by the unique test case that executes each loop body just once. Thus, there exists a test suite TS such that TS satisfies 100% EC but not 100% (E^2C). So (E^2C) gives strictly greater coverage than EC and NC.

Similarly, the edge pair (6, 2), (2, 3) which gives at least 2 executions of the outer loop body is not satisfied by the unique test case that executes each loop body just once. This is a second different reason why (E^2C) gives strictly greater coverage than EC and NC.

Furthermore, the edge pair (1, 2), (2, 7) (which gives 0 executions of both loop bodies) is not satisfied by the unique test case that executes each loop body just once. This is a third different reason why (E^2C) gives strictly greater coverage than EC and NC.

Notice that each of these edge pairs covers a different number of loop executions from P , so genuinely new code executions are being covered.

- (f) (5 points) Give a JML specification (requires, ensures) that accurately describes the black-box functional behaviour of your condensation graph. Using this specification, give test cases and predicated outputs for each of your edge pair requirements in part (e).

Answer:
requires true

```
ensures ( \old(x) >= 0 & \old(y) >= 0 ==> \result = \old(x) * \old(y) &  
( \old(x) < 0 or \old(y) < 0 ==> \result = 0)
```

Note the use of \old(y) in ensures since y is modified during execution

Using this requirement model:

TC1 could be x = 2, y = 1, output = 2.

TC2 could be x = 1, y = 2, output = 2

TC3 could be x = 1, y = 0, output = 0